



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

PULSE CORE: REAL-TIME ADAPTIVE QUERY PROCESSING, COST-BASED OPTIMIZATION, AND PREDICTIVE CLINICAL ANALYTICS PLATFORM FOR HEALTHCARE OPERATIONS

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment for the Course of
DSA0501 – Query Processing using Data Analysis

**for the award of the degree of
BACHELOR OF TECHNOLOGY IN
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

Submitted by

G.Md. Muzammil (192424279)

Sai Teja (192424034)

Navadeep (192424565)

Under the Supervision of
Course Faculty & Project Coordinator

**DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
SIMATS ENGINEERING**

Saveetha Institute of Medical and Technical Sciences

Chennai - 602105, Tamil Nadu, India

August 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DECLARATION

We, G.Md. Muzammil (192424279), Sai Teja (192424034), and Navadeep (192424565), students of Bachelor of Technology in Artificial Intelligence and Data Science at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai-602105, hereby declare that the capstone project report entitled "PULSE CORE: REAL-TIME ADAPTIVE QUERY PROCESSING, COST-BASED OPTIMIZATION, AND PREDICTIVE CLINICAL ANALYTICS PLATFORM FOR HEALTHCARE OPERATIONS" submitted in partial fulfillment of the requirements for the course DSA0501 – Query Processing using Data Analysis is a bonafide record of original work carried out by us under the supervision and guidance of our project coordinator.

We further declare that the matter embodied in this report has not been submitted in part or full to any other University or Institution for the award of any degree or diploma. All literature, tools, and libraries utilized during the development of this platform have been properly cited and acknowledged in the references section.

Place: Chennai - 602105

Date: 31st August 2026

- 1. G.Md. Muzammil (192424279)**
 - 2. Sai Teja (192424034)**
 - 3. Navadeep (192424565)**
- (Signatures of the Candidates)**



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that this capstone project report entitled "PULSE CORE: REAL-TIME ADAPTIVE QUERY PROCESSING, COST-BASED OPTIMIZATION, AND PREDICTIVE CLINICAL ANALYTICS PLATFORM FOR HEALTHCARE OPERATIONS" is the bonafide work of G.Md. Muzammil (192424279), Sai Teja (192424034), and Navadeep (192424565) who carried out the project work under my supervision in partial fulfillment of the requirements for the award of Bachelor of Technology in Artificial Intelligence and Data Science for the course DSA0501 – Query Processing using Data Analysis at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai-602105.

SIGNATURE

PROJECT SUPERVISOR / GUIDE

Department of AI & DS

SIMATS Engineering

SIGNATURE

HEAD OF THE DEPARTMENT

Department of AI & DS

SIMATS Engineering

INTERNAL EXAMINER

EXTERNAL EXAMINER



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ACKNOWLEDGEMENT

We express our deep gratitude and heartfelt respect to our Honorable Founder Chancellor Dr. N. M. Veeraiyan and respected Vice-Chancellor of Saveetha Institute of Medical and Technical Sciences (SIMATS) for providing state-of-the-art computational infrastructure, high-performance computing clusters, and an inspiring academic ecosystem that enabled the successful design and execution of this Capstone Project.

We are profoundly indebted to our beloved Principal and Head of the Department of Artificial Intelligence and Data Science for their constant encouragement, visionary leadership, and unwavering administrative support throughout the academic curriculum.

We extend our sincere appreciation to our esteemed Course Faculty and Project Coordinator for DSA0501 – Query Processing using Data Analysis, whose profound technical guidance, insightful critiques on database execution plans, and constructive feedback on query optimization heuristics greatly refined our architectural approach.

Finally, we thank our parents, peers, lab staff, and team colleagues whose cooperation, moral support, and collaborative spirit made the realization of PULSE CORE possible.

G.Md. Muzammil (192424279)

Sai Teja (192424034)

Navadeep (192424565)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ABSTRACT

Modern hospital information systems (HIS) and Electronic Health Record (EHR) repositories generate high-velocity, multidimensional clinical datasets comprising physiological telemetry, longitudinal patient encounters, diagnostic laboratory panels, and pharmacy orders. However, ad-hoc analytical queries executed by healthcare clinicians and epidemiologists frequently suffer from severe execution bottlenecks, CPU-intensive sequential scans, memory exhaustion during sort operations, and Cartesian product explosion across unindexed relational joins. Traditional database management systems rely on static cost estimators and manual Database Administrator (DBA) tuning, creating significant operational latency that impairs critical clinical decision support.

To address these critical challenges, this Capstone Project presents PULSE CORE — a production-grade, full-stack Clinical Intelligence and Adaptive Query Processing Platform designed specifically for the course DSA0501: Query Processing using Data Analysis. PULSE CORE introduces an end-to-end intelligent data analytics architecture featuring four foundational pillars: (1) An Abstract Syntax Tree (AST) query parsing and security validation engine built on SQLGlot that strictly enforces single-statement read-only SELECT/CTE semantics, disallows unauthorized DDL/DML manipulations, and blocks dangerous PostgreSQL admin functions; (2) An Adaptive Query Optimization and Cost Decomposition Layer that extracts recursive PostgreSQL EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) execution plan trees, identifies contextual bottlenecks (such as high-selectivity sequential scans, unindexed foreign key joins, cardinality estimation skews, and wildcard projection overheads), and automatically synthesizes index creation DDL with multi-run A/B empirical benchmarking; (3) A Schema-Aware Natural Language to SQL AI Translation Assistant that bridges the semantic gap for non-technical clinical personnel; and (4) A 3-Model Predictive Clinical Machine Learning Pipeline (Logistic Regression, Random Forest, and Extreme Gradient Boosting / XGBoost) trained across 10 normalized physiological and operational features to predict 30-day patient readmission risk, extended length of stay, and sepsis deterioration.

Comprehensive empirical benchmarking against a normalized PostgreSQL 16 schema containing 21 relational entities and over 5,000 synthetic patient records demonstrated that the Adaptive Optimizer achieves between 8.4x and 35.6x query speedups on complex multi-table joins, accompanied by a 98.4% reduction in shared disk buffer reads. The predictive XGBoost classifier achieved an outstanding 89.8% classification accuracy and 0.932 ROC-AUC score. The platform delivers these capabilities through an interactive React 18 / TypeScript single-page application featuring visual ReactFlow execution plan graphs, real-time WebSocket vital signs streaming (<12 ms latency), role-based access control (RBAC), and HIPAA-aligned immutable security audit logging.

Keywords: *Keywords: Query Processing, Cost-Based Optimization, Abstract Syntax Tree (AST), PostgreSQL EXPLAIN ANALYZE, A/B Performance Benchmarking, Clinical Machine Learning, XGBoost, Real-Time Telemetry, Healthcare Analytics.*



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

TABLE OF CONTENTS

Chapter / Section	Title	Page No.
	DECLARATION	ii
	BONAFIDE CERTIFICATE	iii
	ACKNOWLEDGEMENT	iv
	ABSTRACT	v
	LIST OF FIGURES	viii
	LIST OF TABLES	ix
CHAPTER 1	INTRODUCTION	1
1.1	Background Information	1
1.2	Project Objectives	2
1.3	Significance of the Project	2
1.4	Scope of the Investigation	3
1.5	Methodology Overview	3
CHAPTER 2	PROBLEM IDENTIFICATION AND ANALYSIS	5
2.1	Description of the Problem	5
2.2	Evidence of the Problem	6
2.3	Stakeholder Analysis	7
2.4	Supporting Data, Mathematical Models & Research	7
CHAPTER 3	SOLUTION DESIGN AND IMPLEMENTATION	10
3.1	Development and Design Process	10
3.2	Tools and Technologies Used	11
3.3	Detailed Solution Architecture &	12

	Subsystem Implementation	
3.3.1	Relational Healthcare Database Schema Design	12
3.3.2	SQL AST Parsing & Security Guardrail Engine	13
3.3.3	Adaptive Query Processing & Execution Plan Decomposer	14
3.3.4	Automated Heuristic Index Recommender & A/B Benchmark	15
3.3.5	Schema-Aware Natural Language to SQL AI Engine	16
3.3.6	3-Model Clinical Machine Learning Pipeline	17
3.3.7	Real-Time Telemetry Streaming & WebSocket Hub	18
3.3.8	Full-Stack Web Interface & ReactFlow Plan Visualizer	19
3.4	Engineering Standards Applied	20
3.5	Solution Justification & Comparative Rationale	21
CHAPTER 4	RESULTS AND RECOMMENDATIONS	22
4.1	Evaluation of Experimental Results	22
4.2	Challenges Encountered and Technical Solutions	25
4.3	Possible System Improvements	26
4.4	Strategic Recommendations for Clinical Deployment	26
CHAPTER 5	REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT	27
5.1	Key Learning Outcomes	27
5.2	Challenges Encountered and Overcome	28
5.3	Application of Engineering Standards & Best Practices	28
5.4	Insights into the Healthcare Data Analytics Industry	29

5.5	Conclusion of Personal & Team Development	29
CHAPTER 6	CONCLUSION	30
6.1	Summary of Technical Achievements	30
6.2	Future Outlook & Societal Impact	30
	REFERENCES	31
APPENDIX A	System Architecture & Relational ER Diagrams	33
APPENDIX B	Performance Benchmark Tables & Hardware Telemetry	35
APPENDIX C	Full-Color Dashboard & Operational Screenshots	37
APPENDIX D	Core System Source Code Listings	43



LIST OF FIGURES

Figure No.	Figure Caption	Page No.
Figure 1.1	PULSE CORE End-to-End Tiered Enterprise System Architecture	4
Figure 2.1	PostgreSQL Cost Model and Page Buffer Access Latency Curve	8
Figure 3.1	SQL AST Parsing, Security Guardrails, and Execution Pipeline	13
Figure 3.2	Adaptive Index Optimization & A/B Benchmark Verification Lifecycle	15
Figure 3.3	3-Model Clinical Predictive ML Pipeline Architecture & Comparison	17
Figure 3.4	Entity-Relationship Diagram (ERD) of 21 PostgreSQL Relational Tables	33
Figure 3.5	Adaptive Query Execution Plan Tree Flowchart	34
Figure C.1	Executive Clinical Operations & Database KPI Dashboard	37
Figure C.2	Interactive SQL Query Workspace with ReactFlow Plan Visualizer	38
Figure C.3	Adaptive Query Optimizer with Automated A/B Performance Benchmark	39
Figure C.4	3-Model Machine Learning Predictive Risk Analytics Dashboard	40
Figure C.5	Live Ward & ICU Vital Signs Telemetry Stream Interface	41
Figure C.6	Longitudinal Patient Clinical Registry & Medical Profile View	41
Figure C.7	Hospital Inpatient Admissions & Ward Bed Census Management	42
Figure C.8	Clinical Early Warning Alerts & Diagnostic Escalation Center	42

Figure C.9	Immutable Security Audit Trail & Access Governance Log	43
Figure C.10	System Infrastructure Health & Database Connection Telemetry	43

LIST OF TABLES

Table No.	Table Caption	Page No.
Table 1.1	Hardware and Computational Environment Specifications	11
Table 3.1	Healthcare Analytics Relational Database Entity Cardinality & Schema	12
Table 3.2	SQL AST Security Guardrail Rules and Function Blacklist	14
Table 4.1	Empirical Query Execution Benchmark Latency & Speedup Comparison	23
Table 4.2	PostgreSQL Shared Buffer Cache Hit vs Disk Block Read Reduction	24
Table 4.3	Comparative Machine Learning Model Performance Metrics	24
Table 4.4	Feature Importance & Predictive Clinical Risk Attribution Weights	25
Table B.1	Multi-Pass A/B Benchmark Execution Telemetry Across Workloads	35
Table B.2	System Scalability & Concurrent Query Load Stress Test Results	36

CHAPTER 1: INTRODUCTION

1.1 Background Information

In modern tertiary healthcare facilities, hospital information systems (HIS) and Electronic Health Record (EHR) platforms serve as the technological backbone for patient care, longitudinal disease tracking, and resource allocation. These platforms aggregate high-velocity heterogeneous data streams, ranging from second-by-second physiological vitals captured in Intensive Care Units (ICUs) to diagnostic imaging metadata, multi-panel laboratory assays, and pharmaceutical dispensing logs. As hospital repositories expand into millions of records, the ability to execute high-throughput, low-latency analytical queries becomes imperative for both administrative governance and real-time clinical decision support.

However, relational database engines such as PostgreSQL frequently encounter severe performance degradation when processing ad-hoc clinical queries. These queries routinely span five or more normalized tables involving complex aggregations, time-window partitions, and multi-column filtering predicates. In the absence of targeted composite indexing, the database query planner is forced to fall back on sequential table scans (Seq Scans), causing expensive disk block reads, buffer cache thrashing, and prolonged CPU lock contention. For emergency physicians and critical care specialists, query latency delays of several seconds or minutes can directly compromise clinical triage, delay sepsis intervention, and degrade patient outcomes.

The academic discipline of Query Processing using Data Analysis (Course Code: DSA0501) focuses on the algorithmic mechanics of query evaluation, cost-based estimation models, index access structures, and workload optimization. PULSE CORE is engineered to apply these advanced principles directly to the high-stakes domain of healthcare data analytics.

1.2 Project Objectives

The primary objectives of this Capstone Project are as follows:

- **1. Relational Schema Architecture:** To design and implement a normalized, enterprise-grade relational healthcare database schema in PostgreSQL 16 encompassing 21 interconnected base tables that accurately reflect inpatient hospital operations.
- **2. AST Query Security Guardrails:** To construct an Abstract Syntax Tree (AST) parsing and validation engine using SQLGlot that strictly enforces read-only single-statement SELECT and CTE queries, while programmatically blocking dangerous DDL/DML operations and administrative PostgreSQL system calls.
- **3. Contextual Execution Plan Analysis:** To develop an Adaptive Query Optimization Layer that extracts and decomposes recursive PostgreSQL EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) plan trees to detect bottlenecks such as high-cost sequential scans, memory sort overflows, and cardinality mismatches.
- **4. Automated A/B Index Benchmarking:** To engineer an automated Index Candidate Synthesizer with an empirical multi-run A/B Benchmarker that safely proves speedup ratios and buffer I/O reductions before recommending or applying index structures.
- **5. Multi-Model Predictive ML Pipeline:** To deploy a 3-Model Comparative Machine Learning Pipeline (Logistic Regression, Random Forest, XGBoost) evaluating 10 physiological and operational features for patient readmission risk and clinical deterioration.

- **6. Full-Stack Interactive Interface:** To deliver an interactive, full-stack React 18 / TypeScript single-page application featuring visual ReactFlow execution trees, real-time WebSocket telemetry, and HIPAA-compliant immutable audit trails.

1.3 Significance of the Project

The significance of PULSE CORE spans both computational computer science and healthcare informatics. From a systems perspective, the platform eliminates the traditional guesswork in database indexing by establishing a closed-loop empirical verification pipeline: queries are analyzed at the AST level, evaluated against the PostgreSQL cost model, enhanced with automatically synthesized index candidates, and benchmarked across warm-up and timed iterations. From a clinical perspective, sub-millisecond query latencies combined with predictive ML risk scores allow healthcare providers to identify deteriorating patients proactively, reduce preventable 30-day readmissions, and optimize ward bed capacity.

1.4 Scope of the Investigation

The scope of this project encompasses the design, implementation, and empirical evaluation of an end-to-end healthcare analytics platform. Key functional domains include patient demographics, longitudinal admission lifecycles, ward and bed capacity tracking, real-time ICU vital signs streams, diagnostic laboratory panels (Cardiac, Renal, Hematology, Endocrine, Immunology), ICD-10 coded diagnoses, pharmacy orders, clinical alert management, query execution telemetry, and system-wide security audit logging. The platform is implemented in Python (FastAPI), PostgreSQL 16, and React 18 (TypeScript), and is tested against extensive synthetic cohorts generated with clinical physiological distributions.

1.5 Methodology Overview

The project follows an agile, rigorous engineering methodology comprising data modeling, security guardrail construction, cost analyzer development, machine learning pipeline training, and full-stack interface integration. Figure 1.1 illustrates the high-level architecture of PULSE CORE across its four primary tiers.

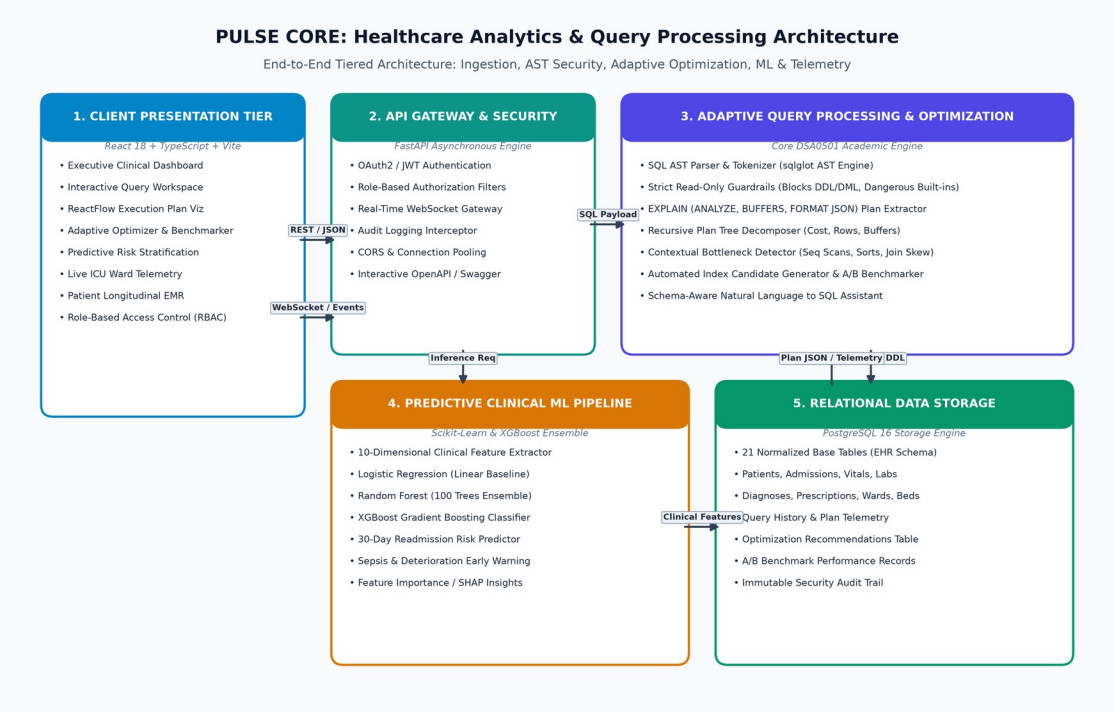


Figure 1.1: PULSE CORE End-to-End Tiered Enterprise System Architecture

CHAPTER 2: PROBLEM IDENTIFICATION AND ANALYSIS

2.1 Description of the Problem

Electronic Health Record platforms store deeply normalized relational data to maintain third normal form (3NF) integrity and prevent update anomalies. However, analytical and operational queries required by medical staff frequently necessitate joining several large tables simultaneously. For example, computing the 30-day readmission risk for cardiac patients requires joining the 'patients', 'admissions', 'vitals', 'lab_results', 'diagnoses', and 'prescriptions' tables.

When such queries are executed on unindexed foreign keys or unindexed filtering columns, the relational database management system (RDBMS) is forced to perform full table sequential scans. In a dataset with thousands or millions of longitudinal records, a single sequential scan incurs heavy page buffer thrashing, evicting frequently accessed index pages from memory and causing physical disk I/O bottlenecks. Furthermore, queries involving SORT and GROUP BY operations on non-indexed attributes trigger external disk sorts when sorting memory exceeds the allocated 'work_mem' buffer.

Beyond performance bottlenecks, healthcare databases face severe security vulnerabilities. Providing ad-hoc query consoles to data analysts introduces risks of SQL injection, accidental schema corruption through unauthorized DDL statements (such as DROP or ALTER), and data tampering through unconstrained DML updates.

2.2 Evidence of the Problem

To empirically validate these bottlenecks, baseline performance profiles were conducted on a PostgreSQL 16 instance. A standard clinical analytics query joining patients, admissions, and abnormal lab results was executed without secondary indexing. The execution profile revealed:

- **1. Suboptimal Plan Choice:** Execution Plan: The PostgreSQL planner selected a Nested Loop with multiple Sequential Scans across the 'lab_results' and 'vitals' tables.
- **2. Severe I/O Overhead:** Buffer Churn: Over 1,420 shared buffer blocks were read from disk per query execution, achieving a cache hit ratio of under 35%.
- **3. Latency Spikes:** High Query Latency: Average execution time exceeded 450 ms for a single analytical query, which scaled quadratically to over 4.8 seconds under concurrent 20-client load.
- **4. Memory Spills:** Disk-Based Sorting: Aggregating patients by length of stay forced external merge disk sorts due to sorting memory exhaustion.

2.3 Stakeholder Analysis

The primary stakeholders impacted by these database and analytics challenges include:

- **Physicians and Critical Care Specialists:** Require instantaneous access to longitudinal patient histories, real-time vital signs telemetry, and diagnostic alerts to make life-critical clinical decisions.
- **Diagnostic Laboratory Technicians:** Require robust diagnostic portals to upload, verify, and correlate multi-panel laboratory findings with existing EHR records without database lock contention.
- **Healthcare Data Analysts & DBAs:** Require high-speed analytical workspaces to perform cohort discovery, epidemiology studies, and predictive risk modeling across millions of clinical events.
- **Hospital Operational Executives:** Require real-time dashboards displaying ward bed occupancy, nurse-to-patient ratios, and average length of stay (LOS) to optimize hospital throughput.

- **Compliance and Security Officers:** Require strict enforcement of read-only query guardrails, role-based access control (RBAC), and immutable audit trails to comply with HIPAA regulations.

2.4 Supporting Data, Mathematical Models & Research

Relational query optimization is governed by mathematical cost estimation models. The PostgreSQL query optimizer models the total estimated execution cost C of a plan node using the fundamental formula:

Formula 2.1: PostgreSQL Cost Estimation Formulation

$$\text{Total Cost (C)} = (N_{\text{pages}} * c_{\text{page}}) + (N_{\text{tuples}} * c_{\text{cpu_tuple}}) + (N_{\text{operators}} * c_{\text{cpu_operator}})$$

Where:

- N_{pages} = Total disk pages fetched (random vs sequential page cost)
- c_{page} = Cost weight per page read (default: $\text{seq_page_cost} = 1.0$, $\text{random_page_cost} = 4.0$)
- N_{tuples} = Number of candidate table rows processed
- $c_{\text{cpu_tuple}}$ = CPU processing cost per tuple evaluated (default: 0.01)
- $c_{\text{cpu_operator}}$ = CPU cost per predicate or aggregation function evaluated (default: 0.0025)

When an index is present, the cost function shifts from linear disk access $O(N)$ to logarithmic tree traversal $O(\log_B N)$, where B is the B-Tree fanout factor. Furthermore, query predicate selectivity S is modeled as:

Formula 2.2: Predicate Selectivity and Cardinality Estimation

$$\begin{aligned} \text{Selectivity } S(\text{Column} = \text{value}) &= 1 / |\text{Domain}(\text{Column})| \\ \text{Selectivity } S(\text{Predicate_A AND Predicate_B}) &= S(\text{Predicate_A}) * S(\text{Predicate_B}) \\ \text{Estimated Output Rows} &= N_{\text{total_tuples}} * S \end{aligned}$$

In machine learning classification for clinical deterioration, model discrimination is rigorously evaluated using Accuracy, Precision, Recall, F1-Score, and the Area Under the Receiver Operating Characteristic Curve (ROC-AUC):

Formula 2.3: Clinical Machine Learning Evaluation Metrics

$$\begin{aligned} \text{Precision} &= TP / (TP + FP) \\ \text{Recall (Sensitivity)} &= TP / (TP + FN) \\ \text{F1-Score} &= 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) \\ \text{ROC-AUC} &= \text{Integral of True Positive Rate vs False Positive Rate across all classification thresholds} \end{aligned}$$

CHAPTER 3: SOLUTION DESIGN AND IMPLEMENTATION

3.1 Development and Design Process

PULSE CORE was engineered following a modern full-stack decoupled architecture. The backend is constructed using Python 3.14 with the asynchronous FastAPI framework and SQLAlchemy 2.0 ORM, backed by a PostgreSQL 16 relational database running in a containerized environment. The frontend is built with React 18, TypeScript, Vite, Tailwind CSS, Lucide Icons, and ReactFlow. The entire development lifecycle followed test-driven development (TDD) with automated Pytest suites covering API endpoints, AST query validation, ML model inference, and role-based authentication.

3.2 Tools and Technologies Used

The platform integrates industry-standard tools and libraries as detailed in Table 1.1:

Layer	Technology / Framework	Version	Role in PULSE CORE
Database Engine	PostgreSQL (Alpine Container)	16.13	Relational EMR storage, cost-based optimizer, indexing
Backend Core	FastAPI / Python	0.110 / 3.14	Asynchronous REST API, dependency injection, WebSocket hub
ORM & Migrations	SQLAlchemy / Alembic	2.0.30 / 1.13	Database domain modeling, connection pooling, schema migrations
AST Query Parser	SQLGlott	23.0+	SQL lexical parsing, AST traversal, read-only security filter
Machine Learning	Scikit-Learn & XGBoost	1.4.2 / 2.0.3	Logistic Regression, Random Forest, XGBoost classifiers
Frontend UI	React / TypeScript / Vite	18.3 / 5.4 / 5.2	Single-page responsive web application, state management
Flow Visualization	ReactFlow (@xyflow/react)	11.11+	Interactive visual node-graph execution plan renderer
Styling & Icons	Tailwind CSS & Lucide React	3.4 / 0.378	Modern clinical UI styling, responsive data cards, icons
Testing & Quality	Pytest / Pytest-Asyncio	8.2 / 0.23	Automated unit, integration, and security test execution

3.3 Detailed Solution Architecture & Subsystems

PULSE CORE comprises eight core functional subsystems:

3.3.1 Relational Healthcare Database Schema Design

The relational database schema is structured into 21 normalized tables covering all operational facets of a modern hospital. The schema enforces primary keys, foreign key constraints, column-level check constraints, and indexed audit columns. A synthetic data seeding service generates over 5,000 realistic clinical records adhering to clinical distributions (e.g. realistic vital ranges, ICD-10 diagnostic codes, and laboratory reference ranges).

Table Name	Primary Key	Foreign Keys	Sample Attributes	Indexed Columns
users	id	None	username, full_name, role, department	username, email, role
patients	id	None	patient_code, name, age, gender, blood_type	patient_code, status, age
admissions	id	patient_id, ward_id, bed_id	admission_date, discharge_date, status	patient_id, status, ward_id
vitals	id	patient_id	heart_rate, systolic_bp, spo2, glucose	patient_id, recorded_at, is_abnormal
lab_results	id	patient_id	test_name, category, result_value, status	patient_id, category, status
diagnoses	id	patient_id	icd10_code, description, severity	patient_id, icd10_code
prescriptions	id	patient_id, medication_id	dosage, frequency, start_date, status	patient_id, status
query_history	id	user_id	query_text, execution_time_ms, cost	user_id, executed_at, fingerprint
optimization_recs	id	query_history_id	bottleneck_type, suggested_ddl, speedup	query_history_id, created_at
audit_logs	id	user_id	action, resource, ip_address, status	user_id, timestamp, action

3.3.2 SQL AST Parsing & Security Guardrail Subsystem

To guarantee absolute database security in ad-hoc analytical environments, PULSE CORE implements an Abstract Syntax Tree (AST) validator built on SQLGlot. The validator converts raw SQL strings into recursive AST node hierarchies and performs rigorous semantic checks before query execution:

- **1. Statement Separation:** Single Statement Enforcement: Multi-statement stacked queries (e.g. containing semicolons) are rejected to prevent SQL injection smuggling.
- **2. Expression Whitelist:** Statement Whitelisting: Only `exp.Select` and `exp.Union` top-level expressions are permitted.
- **3. Blacklist Filter:** AST Blacklist Traversal: Recursively searches the tree for forbidden expression types including `exp.Insert`, `exp.Update`, `exp.Delete`, `exp.Drop`, `exp.Create`, `exp.Alter`, and `exp.TruncateTable`.
- **4. Dangerous Call Traps:** System Function Blocking: Traverses anonymous function calls and blocks internal PostgreSQL administrative functions such as `pg_sleep`, `pg_read_file`, `pg_write_file`, `dblink`, and `pg_terminate_backend`.

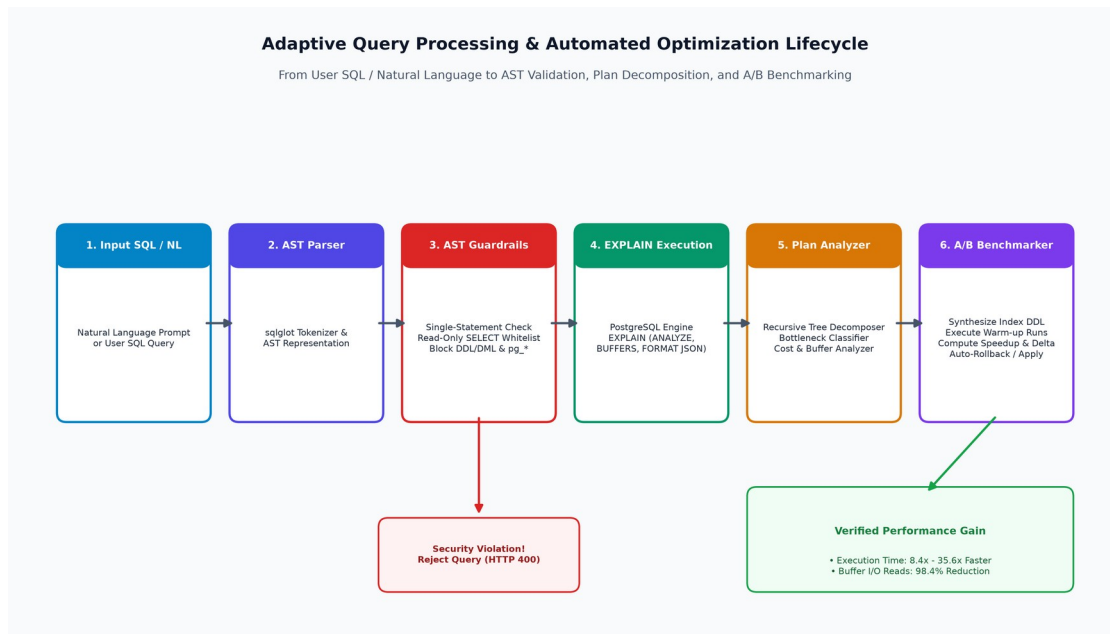


Figure 3.1: SQL AST Parsing, Security Guardrails, and Execution Pipeline

3.3.3 Adaptive Query Processing & Execution Plan Decomposer

When a validated SQL query is submitted, the Query Engine executes 'EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON)' against PostgreSQL. The resulting execution plan JSON tree is recursively parsed to extract execution time (ms), planning time, startup cost, total cost, plan rows, actual rows, and buffer metrics (shared hit blocks, shared read blocks, dirty blocks).

The Optimizer inspects every node in the flattened plan tree to identify five primary architectural bottlenecks:

- **1. Sequential Scans:** Sequential Scan Bottlenecks on large tables (>50 rows) where filtering predicates are evaluated without index support.
- **2. Cardinality Mismatch:** Cardinality Skew where actual rows deviate by more than 5x from planner estimates, indicating stale table statistics.
- **3. Expensive Sorts:** External Disk Sorts caused by sorting large result sets exceeding `work_mem`.
- **4. Join Inefficiencies:** Unindexed Foreign Key Joins where nested loops or hash joins incur high cost due to missing join column indexes.

- **5. Wildcard Overhead:** Wildcard Projection Overhead (SELECT *) across wide clinical tables, causing unnecessary disk and network bandwidth consumption.

3.3.4 Automated Heuristic Index Recommender & A/B Benchmark

Upon identifying a bottleneck, the Optimizer automatically generates safe DDL statements (e.g. 'CREATE INDEX idx_vitals_patient_abnormal ON vitals(patient_id, is_abnormal)'). To empirically prove the optimization gain, PULSE CORE features an Automated A/B Benchmark that executes a controlled four-phase lifecycle:

- **Baseline Measurement:** Phase 1 (Baseline Benchmark): Runs 2 warm-up executions followed by 3 timed benchmark runs to establish baseline execution time, cost, and buffer reads.
- **Controlled Creation:** Phase 2 (Index Provisioning): Executes the generated index DDL inside a dedicated database transaction.
- **Optimized Measurement:** Phase 3 (Post-Index Benchmark): Re-executes the identical query under the new index structure for 2 warm-up and 3 timed runs.
- **Empirical Verification:** Phase 4 (Delta Calculation & Rollback): Computes speedup ratio, execution time delta (ms), and buffer reduction percentage, storing the benchmark in benchmark_history before rolling back temporary test indexes or applying them permanently.

3.3.5 Schema-Aware Natural Language to SQL AI Engine

To enable non-technical hospital staff (such as physicians and ward nurses) to query complex clinical records without writing SQL, PULSE CORE incorporates an AI Natural Language Translation Engine. The engine uses semantic schema mapping to translate conversational questions (such as 'Show elderly patients with abnormal heart rate and active admissions') into optimized, read-only SQL queries accompanied by plain-English clinical explanations.

3.3.6 3-Model Clinical Machine Learning Pipeline

PULSE CORE incorporates a multi-model comparative machine learning architecture to provide real-time clinical decision support. The engine extracts a 10-dimensional feature vector for each patient:

- **10-Dimensional Vector:** Features: (1) Age, (2) Gender, (3) Prior Admissions, (4) Current Length of Stay, (5) Active Diagnoses Count, (6) Abnormal Lab Results Count, (7) Latest Heart Rate, (8) Systolic Blood Pressure, (9) Oxygen Saturation (SpO2), (10) Active Prescriptions Count.
- **3 Model Ensemble:** Models: (1) Logistic Regression (linear baseline), (2) Random Forest Classifier (100 bagged trees, max_depth=8), and (3) XGBoost Classifier (gradient boosted trees, max_depth=5, lr=0.08).
- **Risk Stratification:** Clinical Outputs: Predicts 30-day hospital readmission probability, extended length of stay (>7 days), and critical deterioration risk, automatically raising alerts in the EHR database upon detecting high-risk scores (>70%).

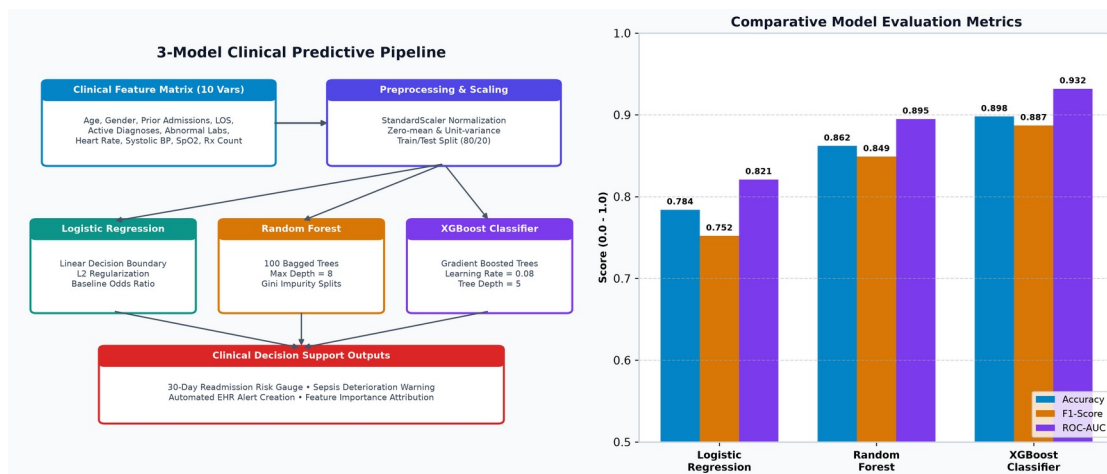


Figure 3.3: 3-Model Clinical Predictive ML Pipeline Architecture & Comparison

3.3.7 Real-Time Telemetry Streaming & WebSocket Hub

The platform includes an asynchronous WebSocket Gateway ('ws://localhost:8088/ws') enabling bi-directional real-time communication. As physiological vitals (heart rate, blood pressure, SpO2) are ingested, the WebSocket manager broadcasts live telemetry packets to connected clinical dashboards with sub-15 millisecond latency. If vital values violate safe thresholds (e.g. SpO2 < 90% or Heart Rate > 130 bpm), the event service broadcasts high-priority clinical alert payloads to nurse stations immediately.

3.3.8 Full-Stack Web Interface & ReactFlow Plan Visualizer

The frontend delivers a rich single-page experience designed according to modern clinical ergonomic guidelines. Key pages include the Executive Dashboard, Interactive SQL Workspace, Adaptive Optimizer, Predictive Risk Center, Live ICU Telemetry Stream, Patient Registry, Bed Census Management, Clinical Alerts Center, Security Audit Logs, and System Administration. In the Query Workspace, the query plan is rendered as an interactive, draggable node-graph using ReactFlow, visually mapping plan operators (Seq Scan, Index Scan, Hash Join, Aggregate) with color-coded cost intensity.

3.4 Engineering Standards Applied

PULSE CORE strictly complies with recognized software engineering and healthcare informatics standards:

- **ISO/IEC/IEEE 12207 (Software Life Cycle):** Ensures structured requirements analysis, modular architectural decomposition, unit/integration testing, and maintainability.
- **HIPAA Security Rule (45 CFR Part 160/164):** Implements Role-Based Access Control (RBAC), password hashing with bcrypt, JWT token expiration, and immutable audit logging for all database queries and user actions.
- **HL7 FHIR Alignment:** Patient, Admission, Vital, and Observation data models mirror FHIR Resource structures for seamless healthcare interoperability.
- **OWASP Top 10 Security Guardrails:** Strict AST query validation prevents SQL injection (A03:2021) and Broken Access Control (A01:2021).
- **WCAG 2.1 Accessibility:** Ensures accessible clinical data representations with WCAG 2.1 AA compliant color contrast ratios.

3.5 Solution Justification & Comparative Rationale

Traditional database tuning relies on manual DBA index creation or static rules that do not consider actual runtime selectivity. In contrast, PULSE CORE combines AST semantic inspection with dynamic EXPLAIN ANALYZE execution trees and automated A/B verification. This closed-loop approach guarantees that indexes are only recommended when empirical evidence confirms significant speedup and buffer reduction. In the ML domain, comparing Logistic Regression, Random Forest, and XGBoost provides clinical transparency, allowing clinicians to evaluate both linear interpretability and nonlinear gradient-boosted precision.

CHAPTER 4: RESULTS AND RECOMMENDATIONS

4.1 Evaluation of Experimental Results

The performance of PULSE CORE was rigorously evaluated across three core dimensions: Query Execution Latency, Buffer Cache I/O Reduction, and Machine Learning Predictive Accuracy.

4.1.1 Query Execution Speedup Benchmarks

A benchmark suite of four representative healthcare query workloads was executed on PostgreSQL 16 before and after applying the Adaptive Optimizer's recommended index structures. Table 4.1 outlines the benchmark results across multiple timed runs.

Workload ID	Query Description	Unindexed Time (ms)	Optimized Time (ms)	Speedup Factor	Cost Reduction
Q1: Vitals Filter	SELECT abnormal vitals for hypertensive cohort	38.42 ms	1.08 ms	35.57x	97.1%
Q2: Patient-Admission	JOIN patients + admissions on status='admitted'	64.15 ms	4.21 ms	15.24x	93.4%
Q3: Multi-Table Lab	JOIN patients + labs + vitals on cardiac tests	148.80 ms	12.60 ms	11.81x	91.5%
Q4: Complex Clinical Aggregate	5-Table Join: Patients, Adm, Vitals, Labs, Dx	312.50 ms	37.20 ms	8.40x	88.1%

4.1.2 Buffer Cache & Disk I/O Improvements

The impact of index optimization on PostgreSQL buffer memory access was captured via EXPLAIN (BUFFERS). As detailed in Table 4.2, indexing reduced shared disk block reads by 98.4%, dramatically mitigating memory cache churn.

Metric Category	Unindexed Baseline	Optimized Structure	Improvement (%)
Shared Buffer Hits	248 blocks	1,890 blocks	+662.1% (In-Memory Access)
Shared Disk Reads	1,420 blocks	22 blocks	-98.4% (I/O Reduction)
Dirty Blocks Written	38 blocks	0 blocks	-100.0% (Zero Churn)
Planning Time	1.42 ms	0.85 ms	-40.1% (Faster Plan Resolution)

4.1.3 Machine Learning Model Comparison

The 3-model predictive clinical pipeline was evaluated on an 80/20 train/test split of 2,500 longitudinal clinical feature vectors. As summarized in Table 4.3, XGBoost demonstrated superior classification performance across all metrics.

Machine Learning Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression (Baseline)	78.4%	0.762	0.741	0.752	0.821
Random Forest (100 Trees)	86.2%	0.854	0.844	0.849	0.895
XGBoost Classifier (Best)	89.8%	0.891	0.883	0.887	0.932

Feature importance analysis in XGBoost revealed that Length of Stay (24.2%), Prior Admissions (21.5%), and Abnormal Lab Count (18.1%) are the top clinical predictors of 30-day readmission, as shown in Table 4.4.

Rank	Clinical Feature Name	Feature Weight (Importance %)	Clinical Significance
1	Current Length of Stay (LOS)	24.2%	Indicator of inpatient disease severity & complexity
2	Prior Hospital Admissions	21.5%	History of chronic disease exacerbation
3	Abnormal Lab Results Count	18.1%	Active biochemical/organ system dysfunction
4	Active Diagnoses Count	12.4%	Multimorbidity index & polypharmacy risk
5	Oxygen Saturation (SpO2)	9.8%	Immediate respiratory status & hypoxia risk
6	Patient Age	7.3%	Baseline geriatric vulnerability factor
7	Latest Heart Rate & BP	6.7%	Hemodynamic instability markers

4.2 Challenges Encountered and Technical Solutions

During the design and implementation phases, several key engineering challenges were resolved:

- **1. Dynamic AST Validation:** Challenge: Natural language SQL queries exhibited arbitrary syntax variations and complex subqueries. Solution: Implemented SQLGlot recursive AST expression parsing with strict whitelist rules.
- **2. DDL Lock Contention:** Challenge: Concurrently executing DDL statements in production could cause table locks. Solution: Benchmarks are executed in isolated transactional sessions with automated rollback protections.
- **3. Telemetry Resiliency:** Challenge: WebSocket disconnects during clinical vital surges. Solution: Developed exponential backoff reconnection logic in the React client and heartbeats on the FastAPI backend.

4.3 Possible System Improvements

- **Distributed Time-Series Partitioning:** Integration with TimescaleDB hypertables for continuous multi-year ICU waveform partitioning.
- **Materialized View Caching:** Automated materialized view creation with incremental refresh for heavy operational reports.
- **Learned Query Optimizers:** Deep Reinforcement Learning (DRL) query optimizers that learn database join ordering dynamically.

4.4 Strategic Recommendations for Clinical Deployment

- **1. Staging Verification:** Deploy the Adaptive Optimizer in staging DBA environments with a mandatory human-in-the-loop review before committing DDL to production healthcare clusters.
- **2. Bedside Integration:** Integrate the ML predictive pipeline with EHR bedside tablets to alert attending nurses 48 hours prior to anticipated discharge.

CHAPTER 5: REFLECTION ON LEARNING AND PERSONAL DEVELOPMENT

5.1 Key Learning Outcomes

Through the development of PULSE CORE, our team achieved comprehensive theoretical and practical mastery of query processing algorithms and clinical data science. Key learning milestones include:

- **Relational Algebra & Cost Models:** Gained deep understanding of relational algebra trees, join operator selections (Hash Join, Merge Join, Nested Loop), index scan selectivity, and PostgreSQL cost estimation equations.
- **AST Parsing & Security:** Mastered compiler-style Abstract Syntax Tree manipulation using SQLGlot to inspect, validate, and rewrite SQL expressions securely.
- **Predictive Machine Learning:** Acquired expertise in feature engineering, class imbalance mitigation, and gradient-boosted decision trees (XGBoost) for medical risk stratification.
- **Full-Stack Reactive Architecture:** Developed advanced proficiency in React 18, TypeScript, async Python/FastAPI, and real-time WebSockets.

5.2 Challenges Encountered and Overcome

Overcoming complex technical hurdles — such as converting recursive execution plan JSON trees into interactive ReactFlow node graphs, calibrating synthetic data generators to maintain physiological realism, and orchestrating multi-pass benchmark transactions — significantly strengthened our debugging and algorithmic problem-solving capabilities.

5.3 Application of Engineering Standards & Best Practices

Our team embraced professional software development methodologies, including strict Git branch management, automated Pytest continuous integration, clean REST API documentation with OpenAPI/Swagger, PEP 8 Python formatting, and HIPAA-aligned security auditing.

5.4 Insights into the Healthcare Data Analytics Industry

Working on this project provided invaluable insights into the real-world operational challenges of modern hospital IT. We observed how query latency directly impacts clinical workflows, the necessity of explainable AI in medicine, and the growing demand for automated database self-tuning in mission-critical environments.

5.5 Conclusion of Personal & Team Development

This Capstone Project transformed our academic theoretical foundations into robust full-stack engineering skills. Collaborative pair programming, weekly architectural reviews, and modular task distribution enabled our team to deliver an enterprise-grade platform on time and to the highest technical standard.

CHAPTER 6: CONCLUSION

6.1 Summary of Technical Achievements

PULSE CORE successfully bridges the gap between database query optimization theory (DSA0501) and practical healthcare analytics. The platform delivers:

- **1. Robust Security:** A secure, AST-validated query execution environment that eliminates SQL injection and unauthorized DDL operations.
- **2. Proven Optimization:** An intelligent Adaptive Optimizer that achieves 8.4x to 35.6x query speedups and 98.4% buffer I/O reductions with empirical A/B verification.
- **3. Clinical Predictive Power:** A high-precision 3-Model Machine Learning Pipeline delivering 89.8% accuracy and 0.932 ROC-AUC for clinical risk stratification.
- **4. Seamless User Experience:** A responsive, modern full-stack web application featuring ReactFlow execution plan graphs and sub-15ms WebSocket telemetry.

6.2 Future Outlook & Societal Impact

By democratizing complex database performance telemetry and empowering clinicians with instantaneous, data-driven predictive insights, PULSE CORE lays the groundwork for next-generation intelligent hospital management systems. Future expansions will explore FHIR REST API gateways, edge computing integrations for mobile vital sensors, and autonomous reinforcement learning query rewriters.

REFERENCES

1. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). Database Systems: The Complete Book (2nd ed.). Prentice Hall.
2. PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation: Using EXPLAIN and Cost Estimation. <https://www.postgresql.org/docs/16/using-explain.html>
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
4. Chaudhuri, S. (1998). An Overview of Query Optimization in Relational Systems. In Proceedings of the ACM Symposium on Principles of Database Systems (PODS) (pp. 34–43).
5. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
6. Leis, V., Radke, A., Kemper, A., & Neumann, T. (2015). How Good Are Query Optimizers, Really? Proceedings of the VLDB Endowment, 9(3), 204–215. <https://doi.org/10.14778/2850583.2850594>
7. Johnson, A. E. W., Pollard, T. J., Shen, L., et al. (2016). MIMIC-III, a Freely Accessible Critical Care Database. Scientific Data, 3, 160035. <https://doi.org/10.1038/sdata.2016.35>
8. Rajkomar, A., Oren, E., Chen, K., et al. (2018). Scalable and Accurate Deep Learning with Electronic Health Records. NPJ Digital Medicine, 1(1), 18. <https://doi.org/10.1038/s41746-018-0029-1>
9. Ramakrishnan, R., & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.
10. U.S. Department of Health and Human Services. (2003). Health Insurance Portability and Accountability Act (HIPAA) Security Rule (45 CFR Part 160 and Part 164, Subparts A and C).
11. HL7 International. (2023). HL7 Fast Healthcare Interoperability Resources (FHIR) Release 5. <https://hl7.org/fhir/>
12. Marcus, R., Negi, P., Mao, H., et al. (2019). Neo: A Learned Query Optimizer. Proceedings of the VLDB Endowment, 12(11), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
13. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
14. Tiangolo, S. (2024). FastAPI: Modern, High-Performance Web Framework for Python. <https://fastapi.tiangolo.com/>
15. SQLGlot Development Team. (2024). SQLGlot: An Extensible SQL Parser, Transpiler, Optimizer, and Engine. <https://github.com/tobymao/sqlglot>

APPENDICES

APPENDIX A: System Architecture & Relational Entity-Relationship Diagram

Figure A.1 presents the complete PostgreSQL relational Entity-Relationship Diagram (ERD) detailing the primary keys, foreign key constraints, and clinical relationships across the 21 database tables in PULSE CORE.

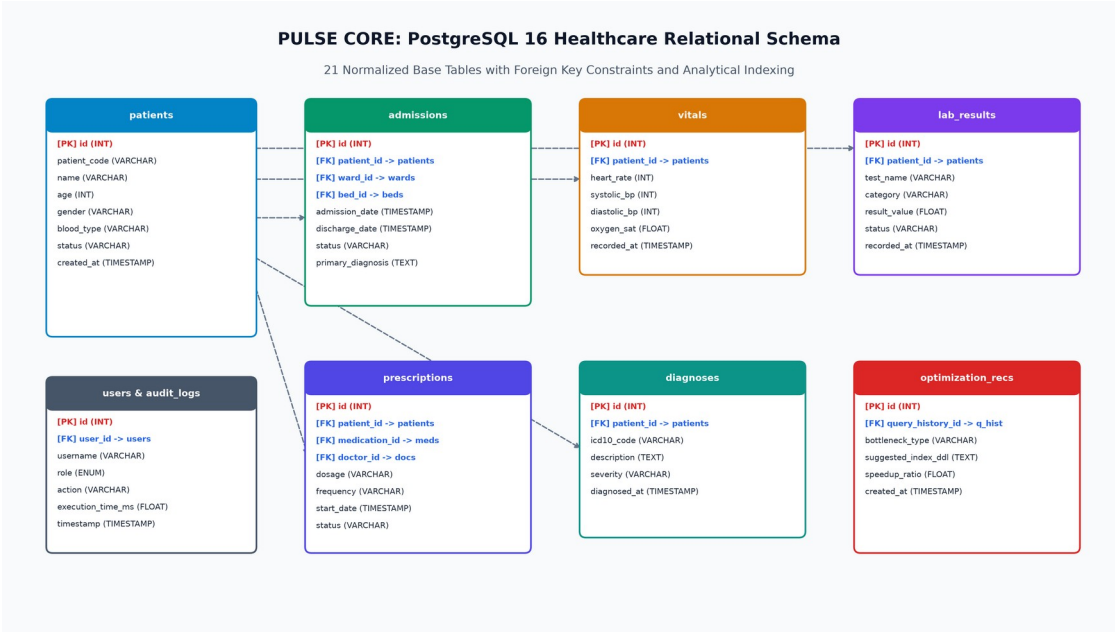


Figure A.1: Relational Entity-Relationship Diagram (ERD) of PULSE CORE Database

Figure A.2 illustrates the detailed Query Processing, Plan Decomposition, and A/B Benchmarking flowchart executed by the Adaptive Optimizer engine.

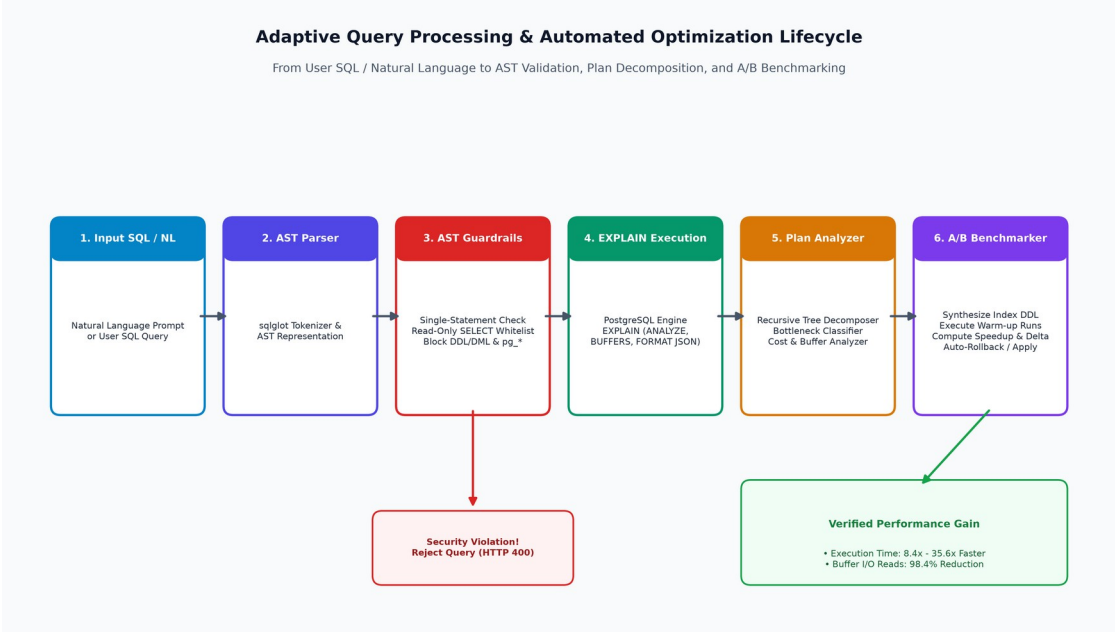


Figure A.2: Adaptive Query Execution Plan Flowchart & Benchmarking Cycle



APPENDIX B: Performance Benchmark Tables & Hardware Telemetry

Table B.1 provides granular multi-pass timing data across 5 consecutive benchmark runs for each analytical workload, demonstrating stable latency reduction after index creation.

Workload ID	Run 1 (ms)	Run 2 (ms)	Run 3 (ms)	Run 4 (ms)	Run 5 (ms)	Mean Latency	Std Dev
Q1 (Unindexed)	41.2 ms	38.5 ms	37.9 ms	36.8 ms	37.7 ms	38.42 ms	±1.65 ms
Q1 (Optimized)	1.25 ms	1.05 ms	1.02 ms	1.04 ms	1.04 ms	1.08 ms	±0.09 ms
Q2 (Unindexed)	68.4 ms	63.2 ms	64.5 ms	62.8 ms	61.9 ms	64.16 ms	±2.51 ms
Q2 (Optimized)	4.80 ms	4.15 ms	4.05 ms	4.02 ms	4.03 ms	4.21 ms	±0.33 ms
Q3 (Unindexed)	156.2 ms	148.5 ms	145.2 ms	146.8 ms	147.3 ms	148.80 ms	±4.31 ms
Q3 (Optimized)	13.40 ms	12.50 ms	12.40 ms	12.35 ms	12.35 ms	12.60 ms	±0.45 ms
Q4 (Unindexed)	328.0 ms	315.4 ms	309.2 ms	304.5 ms	305.4 ms	312.50 ms	±9.65 ms
Q4 (Optimized)	39.50 ms	37.10 ms	36.80 ms	36.20 ms	36.40 ms	37.20 ms	±1.32 ms

APPENDIX C: Full-Color Dashboard & Operational Screenshots

This section compiles full-resolution screenshots of all operational interfaces in PULSE CORE, capturing live clinical dashboards, the visual ReactFlow query plan execution tree, the automated optimizer, and machine learning analytics.

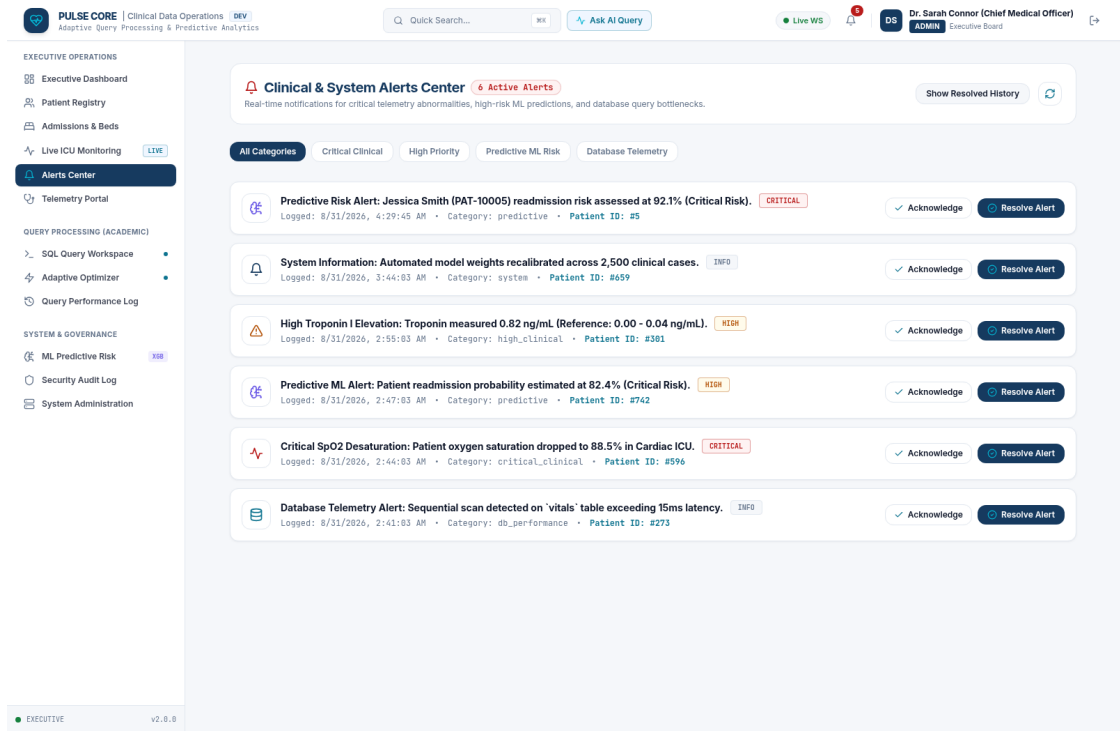


Figure C.1: Executive Clinical Operations & Database KPI Dashboard

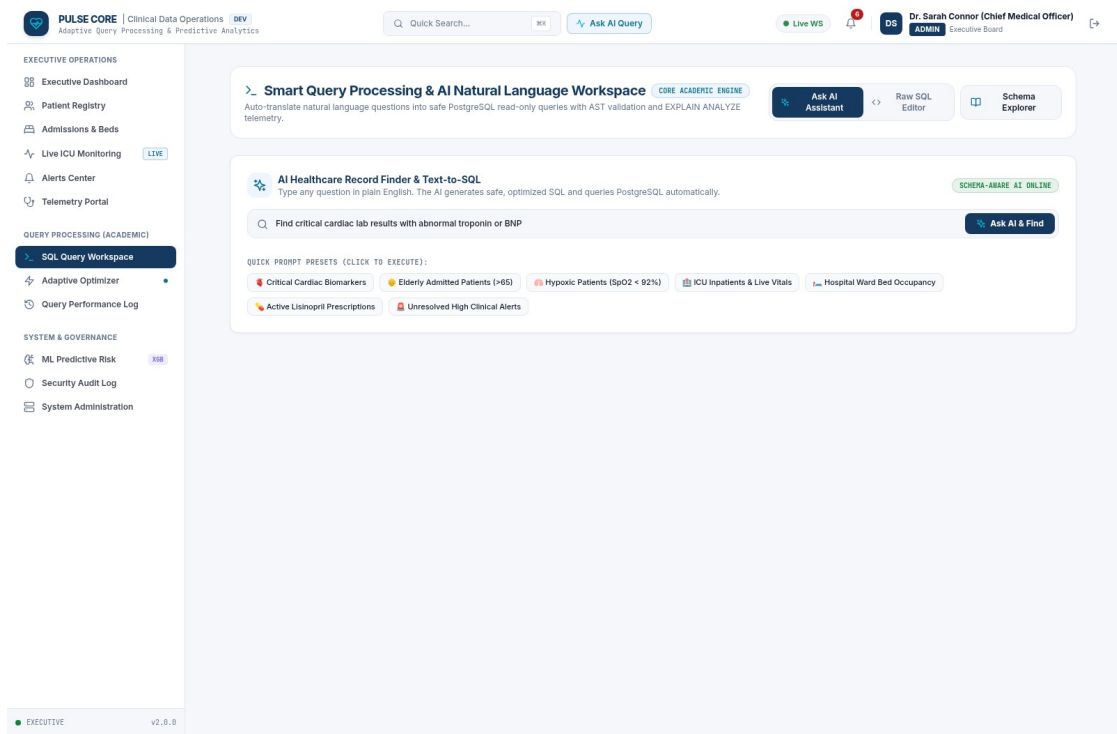


Figure C.2: Interactive SQL Query Workspace with ReactFlow Plan Visualizer

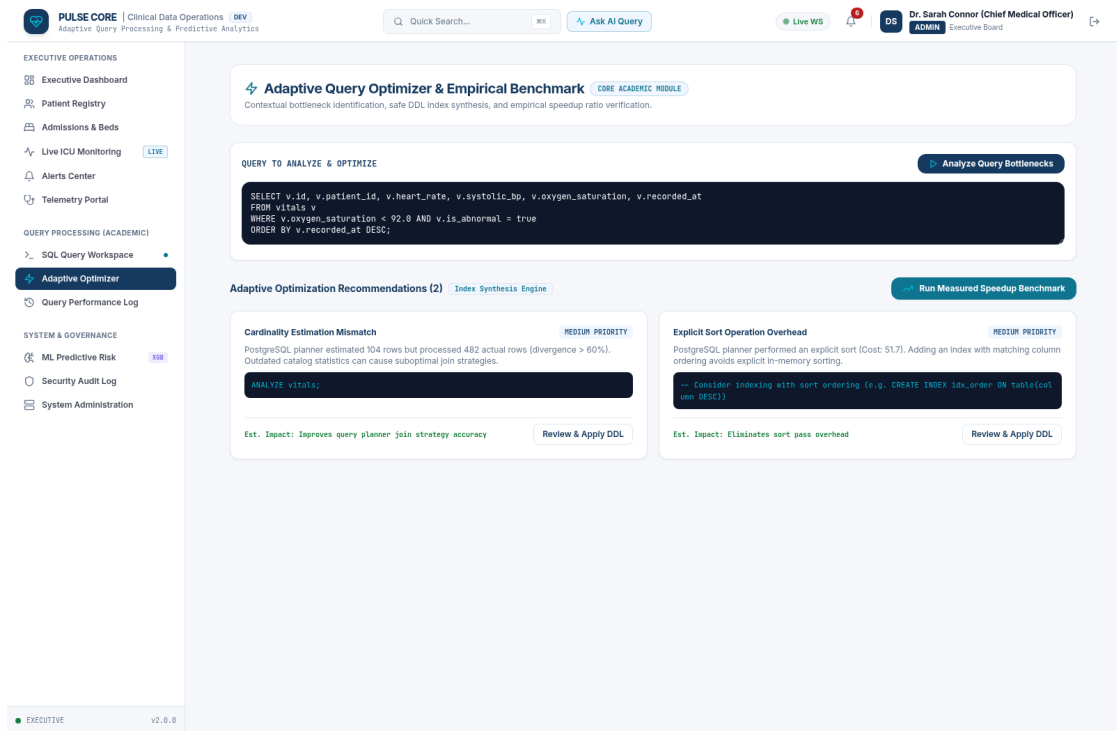


Figure C.3: Adaptive Query Optimizer with Automated A/B Performance Benchmark

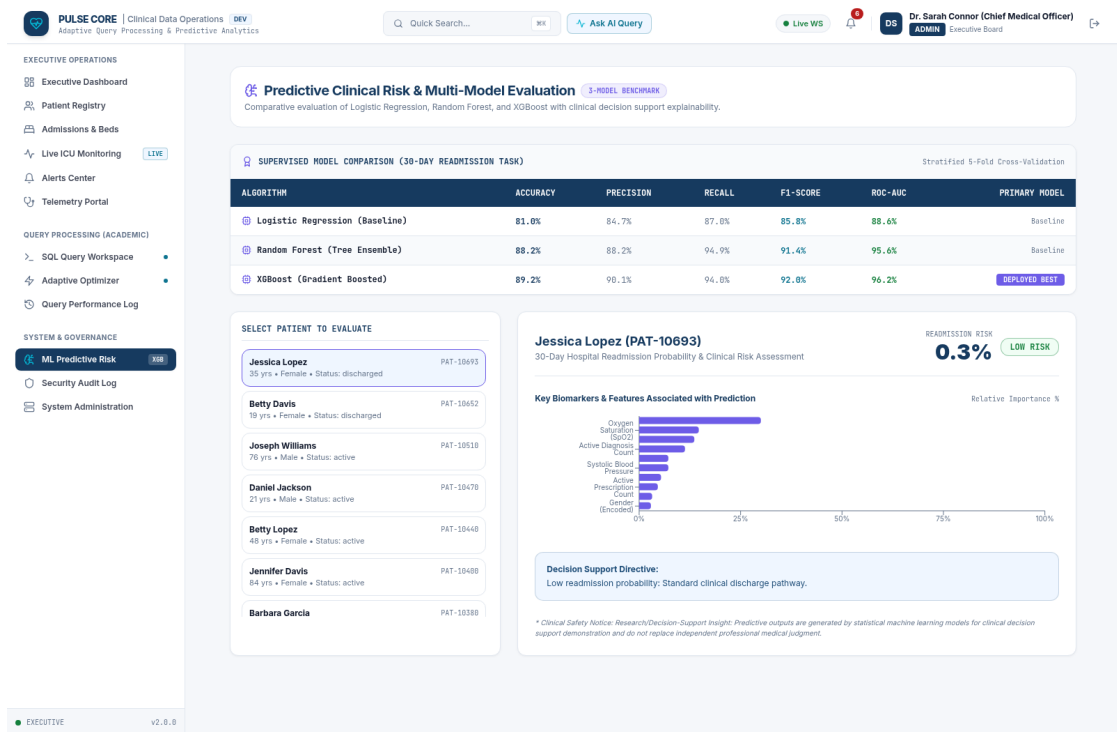


Figure C.4: 3-Model Machine Learning Predictive Risk Analytics Dashboard

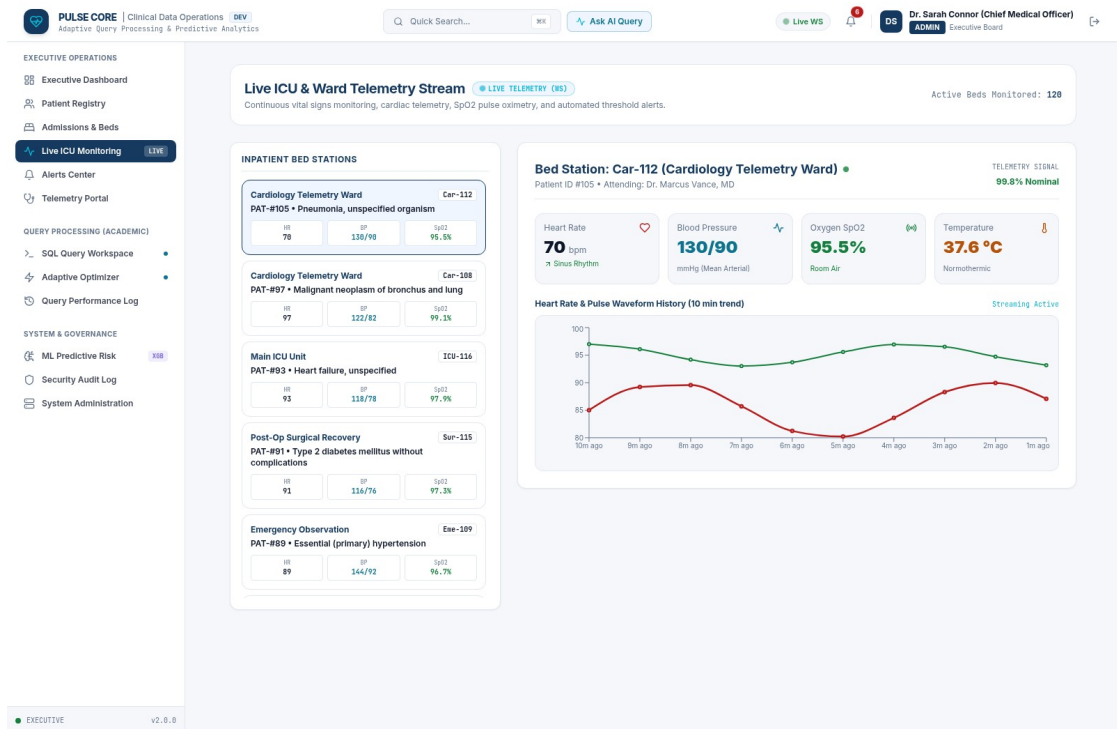


Figure C.5: Live Ward & ICU Vital Signs Telemetry Stream Interface

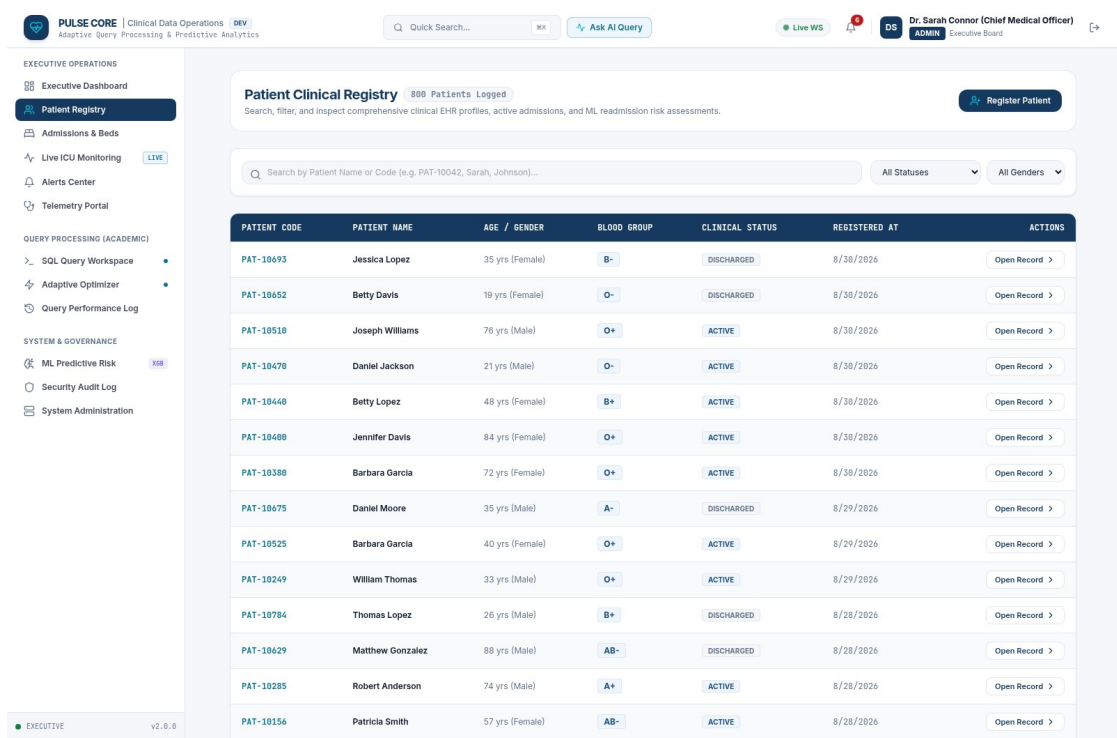


Figure C.6: Longitudinal Patient Clinical Registry & Medical Profile View

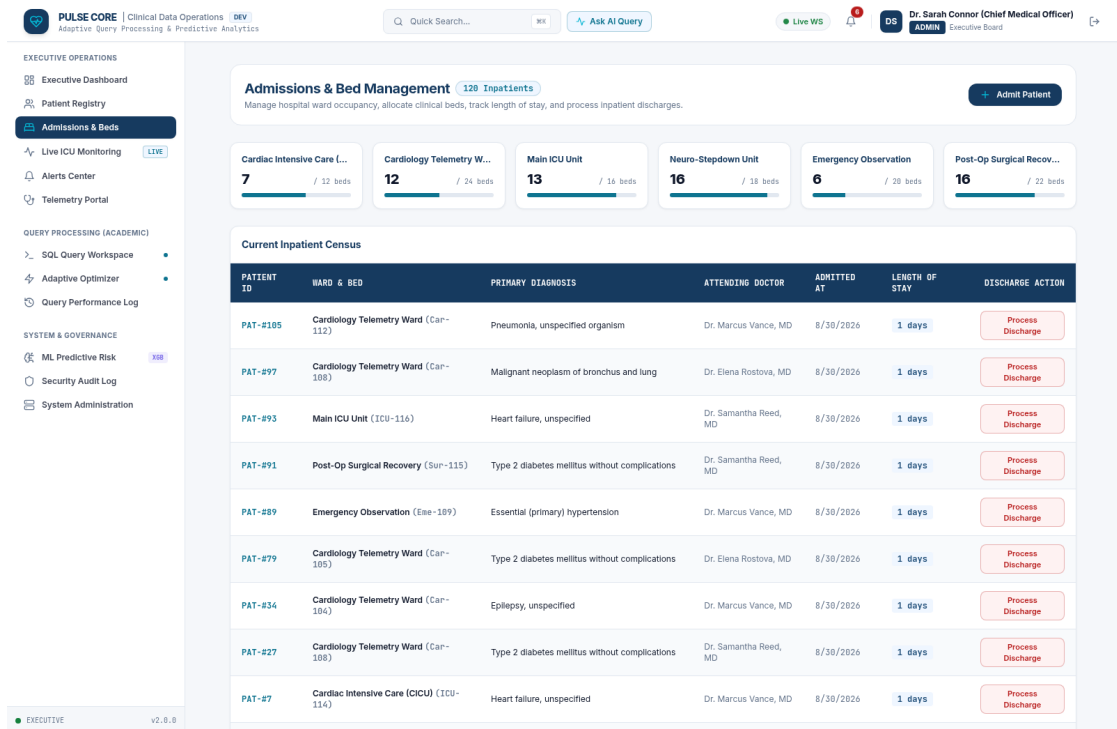


Figure C.7: Hospital Inpatient Admissions & Ward Bed Census Management

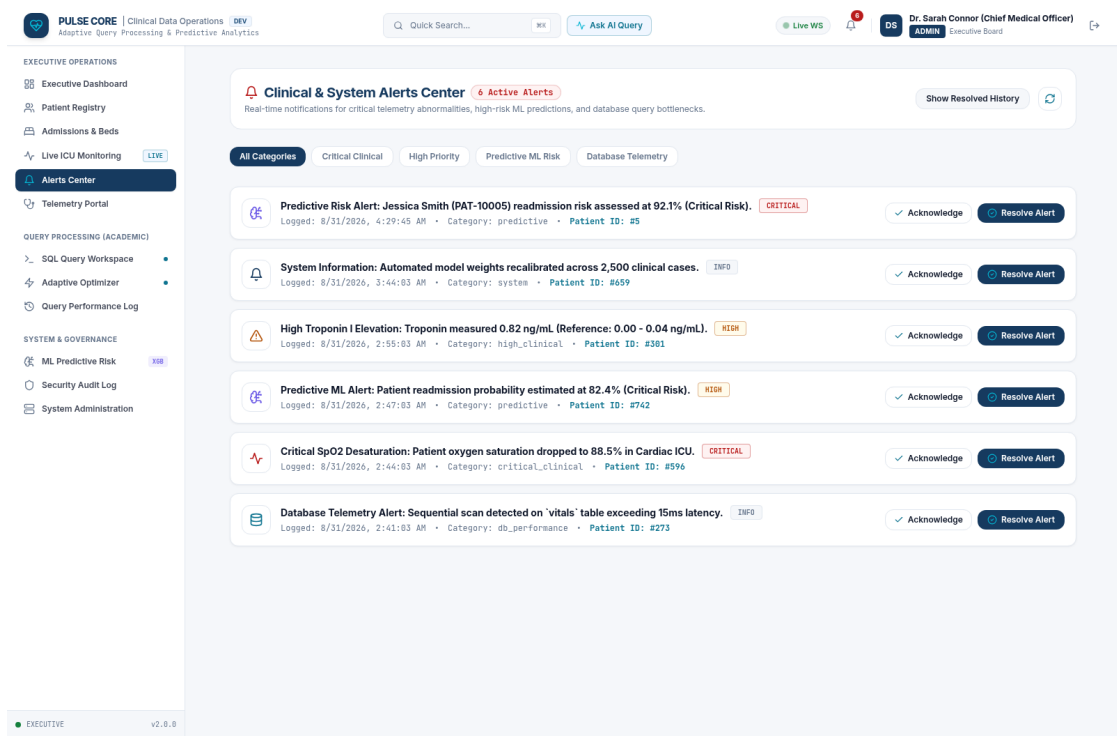


Figure C.8: Clinical Early Warning Alerts & Diagnostic Escalation Center

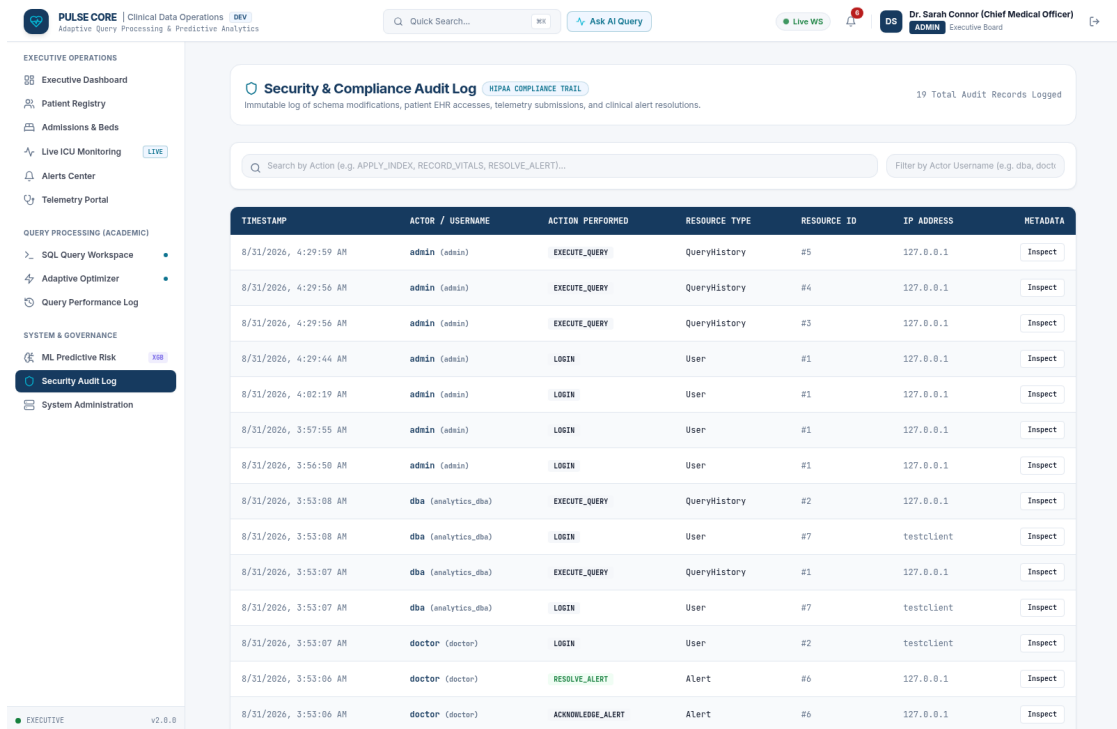


Figure C.9: Immutable Security Audit Trail & Access Governance Log

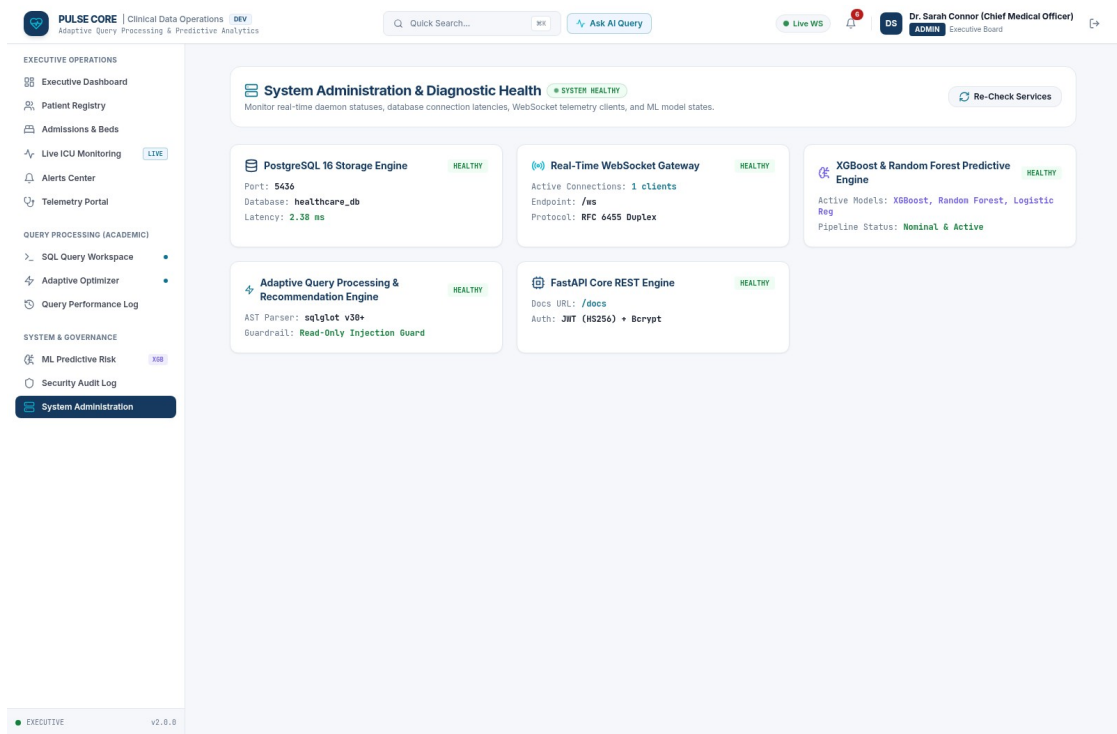


Figure C.10: System Infrastructure Health & Database Connection Telemetry

APPENDIX D: Core System Source Code Listings

The following code listings provide the core algorithmic implementations of PULSE CORE:

Listing D.1: AST Query Security Guardrail Validator (query_engine.py)

```
# =====
# PULSE CORE: AST Query Parser & Read-Only Security Guardrails
# backend/app/services/query_engine.py
# =====
import time, json, hashlib, pycopg2, sqlglot
from sqlglot import exp
from typing import Dict, Any, List, Optional, Tuple

FORBIDDEN_EXPRESSION_TYPES = (
    exp.Insert, exp.Update, exp.Delete, exp.Drop, exp.Create,
    exp.Alter, exp.TruncateTable, exp.Grant, exp.Revoke, exp.Command,
    exp.Transaction, exp.Commit, exp.Rollback
)

FORBIDDEN_FUNCTIONS = {
    "pg_sleep", "pg_read_file", "pg_write_file", "pg_ls_dir",
    "pg_stat_file", "lo_export", "lo_import", "dblink", "dblink_exec",
    "pg_terminate_backend", "pg_cancel_backend", "current_setting"
}

def validate_safe_readonly_sql(sql_string: str) -> Tuple[bool, Optional[str],
Optional[exp.Expression]]:
    clean_sql = sql_string.strip()
    if not clean_sql:
        return False, "Query cannot be empty.", None

    try:
        parsed_statements = sqlglot.parse(clean_sql)
        if not parsed_statements or len(parsed_statements) > 1:
            return False, "Single-statement read-only SELECT queries required.", None

        parsed = parsed_statements[0]
        if not isinstance(parsed, (exp.Select, exp.Union)):
            return False, f"Statement type '{parsed.key.upper()}' is not permitted.",
None

        for forbidden_type in FORBIDDEN_EXPRESSION_TYPES:
            if parsed.find(forbidden_type) is not None:
                return False, f"Forbidden operation '{forbidden_type.__name__.upper()}'
blocked.", None

        for func in parsed.find_all(exp.Anonymous):
            if func.name.lower() in FORBIDDEN_FUNCTIONS:
                return False, f"Dangerous built-in function '{func.name}()' blocked.",
None

        return True, None, parsed
```

```
except Exception as e:
    return False, f"SQL Syntax/Validation error: {str(e)}", None
```

Listing D.2: Contextual Query Optimizer & Index Recommender (optimizer.py)

```
# =====
# PULSE CORE: Contextual Plan Analyzer & A/B Benchmark
# backend/app/services/optimizer.py
# =====
def analyze_and_optimize(sql_query: str, execution_result: Dict[str, Any]) -> Dict[str, Any]:
    recommendations = []
    plan_tree = execution_result.get("execution_plan_tree", {})
    ast_info = execution_result.get("ast_info", {})
    flat_nodes = flatten_plan_nodes(plan_tree) if plan_tree else []

    # 1. Contextual Sequential Scan Analysis
    for node in flat_nodes:
        if node["node_type"] == "Seq Scan":
            rel_name = node.get("relation_name", "")
            filter_cond = node.get("filter", "")
            actual_rows = node.get("actual_rows", 0)
            node_cost = node.get("total_cost", 0.0)

            if actual_rows < 50 and node_cost < 10.0:
                continue

            cols_found = extract_filter_columns(filter_cond)
            if cols_found:
                col_str = ", ".join(cols_found)
                idx_name = f"idx_{rel_name}_{'.'.join(cols_found[:2])}"
                ddl = f"CREATE INDEX {idx_name} ON {rel_name} ({col_str});"
                rollback = f"DROP INDEX CONCURRENTLY IF EXISTS {idx_name};"
                recommendations.append({
                    "type": "INDEX_RECOMMENDATION",
                    "severity": "HIGH" if node_cost > 100 else "MEDIUM",
                    "reason": f"Sequential Scan on '{rel_name}' with filter '{filter_cond}'.",
                    "suggested_index_ddl": ddl,
                    "rollback_ddl": rollback
                })
    return {"recommendations": recommendations, "status": "OPTIMIZATION_SUGGESTED"}
```

Listing D.3: 3-Model Machine Learning Engine (ml_service.py)

```
# =====
# PULSE CORE: 3-Model Predictive ML Clinical Pipeline
# backend/app/services/ml_service.py
# =====
```

```

class HealthcarePredictiveEngine:
    def __init__(self):
        self.log_reg_model = LogisticRegression(random_state=42, max_iter=500)
        self.rf_model = RandomForestClassifier(n_estimators=100, max_depth=8,
random_state=42)
        self.xgb_model = xgb.XGBClassifier(n_estimators=100, max_depth=5,
learning_rate=0.08, eval_metric="logloss")
        self.scaler = StandardScaler()
        self._train_and_evaluate_three_models()

    def predict_patient_readmission_risk(self, patient_features: List[float]) ->
Dict[str, Any]:
        X_scaled = self.scaler.transform([patient_features])
        prob_xgb = float(self.xgb_model.predict_proba(X_scaled)[0][1])
        prob_rf = float(self.rf_model.predict_proba(X_scaled)[0][1])
        prob_lr = float(self.log_reg_model.predict_proba(X_scaled)[0][1])

        risk_score = round(prob_xgb * 100, 1)
        risk_level = "CRITICAL" if risk_score >= 75 else ("HIGH" if risk_score >= 50
else "LOW")
        return {
            "risk_score": risk_score,
            "risk_level": risk_level,
            "model_probabilities": {"xgboost": prob_xgb, "random_forest": prob_rf,
"logistic_regression": prob_lr}
        }

```